TECHNISCHE
UNIVERSITÄT
MÜNCHEN

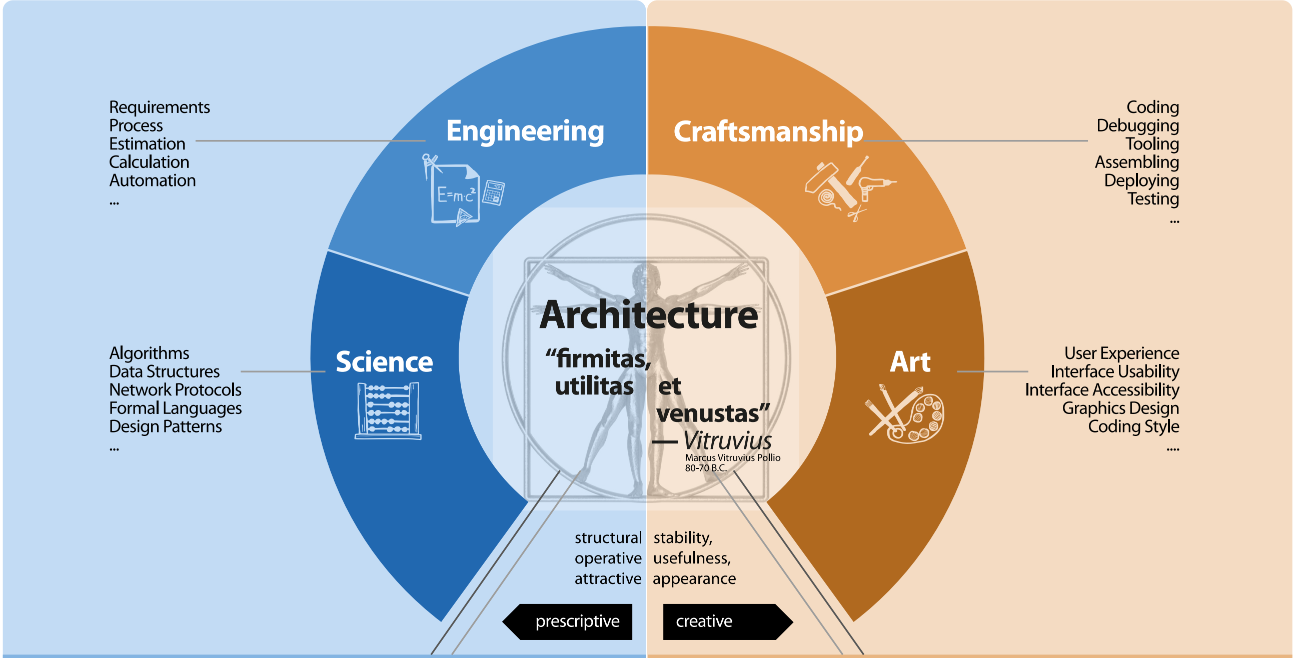
# Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

ARCHITECTURE
FUNDAMENTALS

# Architecture Stargate

TECHNISCHE
UNIVERSITÄT
MÜNCHEN



Requirements
Process
Estimation
Calculation
Automation
...

**Engineering**

**Craftsmanship**

Coding
Debugging
Tooling
Assembling
Deploying
Testing
...

Algorithms
Data Structures
Network Protocols
Formal Languages
Design Patterns
...

**Science**

**Architecture**

**“firmitas,**
**utilitas et**
**venustas”**
— *Vitruvius*
Marcus Vitruvius Pollio
80-70 B.C.

**Art**

User Experience
Interface Usability
Interface Accessibility
Graphics Design
Coding Style
....

structural stability,
operative usefulness,
attractive appearance

prescriptive

creative

**Structural Scientific Definition** of Architecture:
Architecture of a system is the set of fundamental **concepts** and **properties** of the system in its environment, embodied in its **elements**[, **interfaces**], **relationships**[ and **behaviours**] and the principles of its **design** and **evolution**.
(based on industry standard *ISO/IEC 42010*)

**Holistic Artistic Definition** of Architecture:
Great architecture is achieved **harmony** and **accord** of all **parts**, where you no longer can add, modify or remove anything without impairing the **whole**.
(based on a quote by *Leon Battista Alberti* on beauty)

**Architektur** ist nicht einfach zu definieren. Man kann Architektur sowohl **strukturell wissenschaftlich** über messbare Elemente (elements), Schnittstellen (interfaces) und Beziehungen (relationships) als auch **ganzheitlich künstlerisch** über “die Harmonie und den Einklang aller Teile” definieren. Die “Wahrheit” liegt in der Praxis irgendwo dazwischen, denn die beiden Extrema spannen einen breiten Raum auf, in dem alle Lösungen in der Praxis liegen.

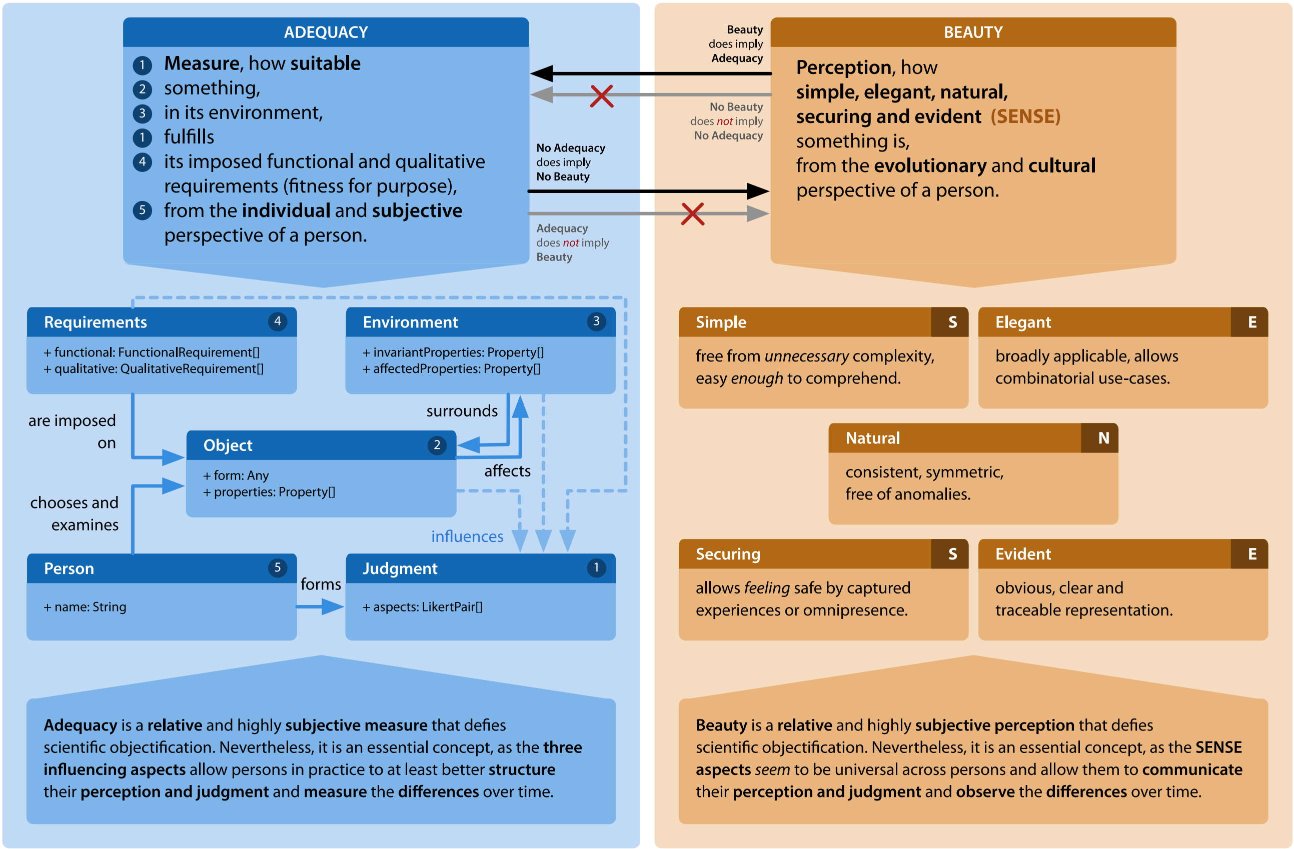
Auf der strukturell wissenschaftlichen Seite definiert sich Architektur über die Aspekte **Science** (insbesondere Computer Science) und **Engineering** (insbesondere Software Engineering). Auf der ganzheitlich künstlerischen Seite definiert sich Architektur über die Aspekte **Craftsmanship** (Handwerk, insbesondere Programmieren) und **Art** (Kunst, insbesondere User Experience).

## Fragen

- Wie kann man **Architektur** definieren?
- Über welche vier **Aspekte** kann man **Architektur** aufspannen?

# Adequacy and Beauty

Intellectual Content: Version 1.0.2 (2022-10-09), Authored 2021-2022 by Dr. Ralf S. Engelschall, based on pre-work of Stefan Brütsch
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.



**ADEQUACY**

① **Measure**, how **suitable**
② something,
③ in its environment,
① fulfills
④ its imposed functional and qualitative requirements (fitness for purpose),
⑤ from the **individual** and **subjective** perspective of a person.

**Beauty** does imply **Adequacy**

**No Beauty** does *not* imply **No Adequacy**

**No Adequacy** does imply **No Beauty**

**Adequacy** does *not* imply **Beauty**

**BEAUTY**

**Perception**, how
**simple, elegant, natural, securing and evident (SENSE)**
something is,
from the **evolutionary** and **cultural** perspective of a person.

**Requirements** ④
+ functional: FunctionalRequirement[]
+ qualitative: QualitativeRequirement[]

**Environment** ③
+ invariantProperties: Property[]
+ affectedProperties: Property[]

**Object** ②
+ form: Any
+ properties: Property[]

**Person** ⑤
+ name: String

**Judgment** ①
+ aspects: LikertPair[]

**Simple** S
free from *unnecessary* complexity, easy *enough* to comprehend.

**Elegant** E
broadly applicable, allows combinatorial use-cases.

**Natural** N
consistent, symmetric, free of anomalies.

**Securing** S
allows *feeling* safe by captured experiences or omnipresence.

**Evident** E
obvious, clear and traceable representation.

**Adequacy** is a **relative** and highly **subjective measure** that defies scientific objectification. Nevertheless, it is an essential concept, as the **three influencing aspects** allow persons in practice to at least better **structure** their **perception and judgment** and **measure** the **differences** over time.

**Beauty** is a **relative** and highly **subjective perception** that defies scientific objectification. Nevertheless, it is an essential concept, as the **SENSE aspects** *seem* to be universal across persons and allow them to **communicate** their **perception and judgment** and **observe** the **differences** over time.

Angemessenheit ist definiert als das Maß, wie geeignet etwas, in seiner Umgebung, die gestellten funktionalen und qualitativen Anforderungen erfüllt (Eignung für den Einsatzzweck), aus der individuellen und subjektiven Perspektive einer Person.

Angemessenheit ist ein relatives und höchst subjektives Maß, das sich einer wissenschaftlichen Objektivierung entzieht. Dennoch ist Angemessenheit ein wichtiges Konzept, da die drei beeinflussenden Aspekte (Anforderungen, Umgebung, Objekt) es den Personen in der Praxis ermöglichen, ihre Wahrnehmung und ihr Urteil zumindest besser zu strukturieren und die Unterschiede über die Zeit zu messen.

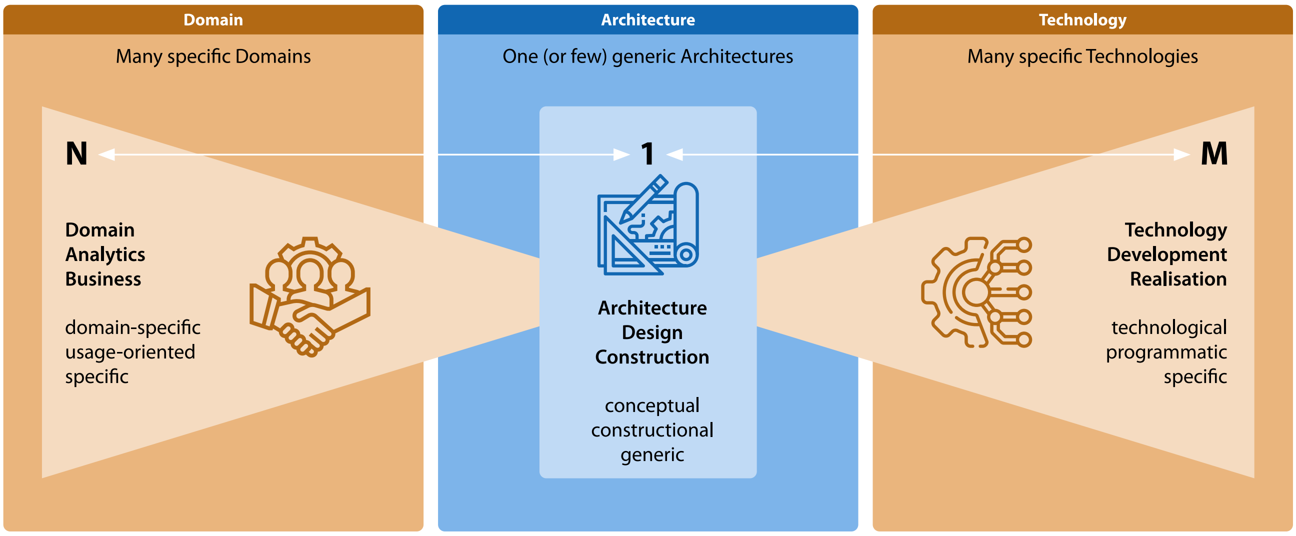
Schönheit ist definiert als die Wahrnehmung, wie einfach, elegant, natürlich, sicher und offensichtlich (SENSE) etwas ist, aus der evolutionären und kulturellen Perspektive eines Menschen.

Schönheit ist eine relative und höchst subjektive Wahrnehmung, die sich einer wissenschaftlichen Objektivierung entzieht. Dennoch ist Schönheit ein wichtiges Konzept, da die SENSE-Aspekte universell über Personen hinweg zu sein scheinen und es ihnen erlauben, ihre Wahrnehmung und ihr Urteil mitzuteilen und die Unterschiede über die Zeit zu messen.

## Fragen

- ? Kann man Angemessenheit oder Schönheit allgemein messen?
- ? Kann man Angemessenheit oder Schönheit im Kontext einer einzelnen Person messen?

# King Discipline Architecture



**Domain**

Many specific Domains

N

**Domain**
**Analytics**
**Business**

domain-specific
usage-oriented
specific

**Architecture**

One (or few) generic Architectures

1

**Architecture**
**Design**
**Construction**

conceptual
constructional
generic

**Technology**

Many specific Technologies

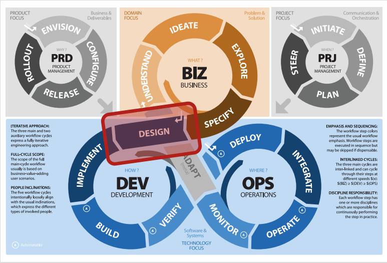
M

**Technology**
**Development**
**Realisation**

technological
programmatic
specific

**Software Engineering Disciplines**

**Software Engineering Workflow**

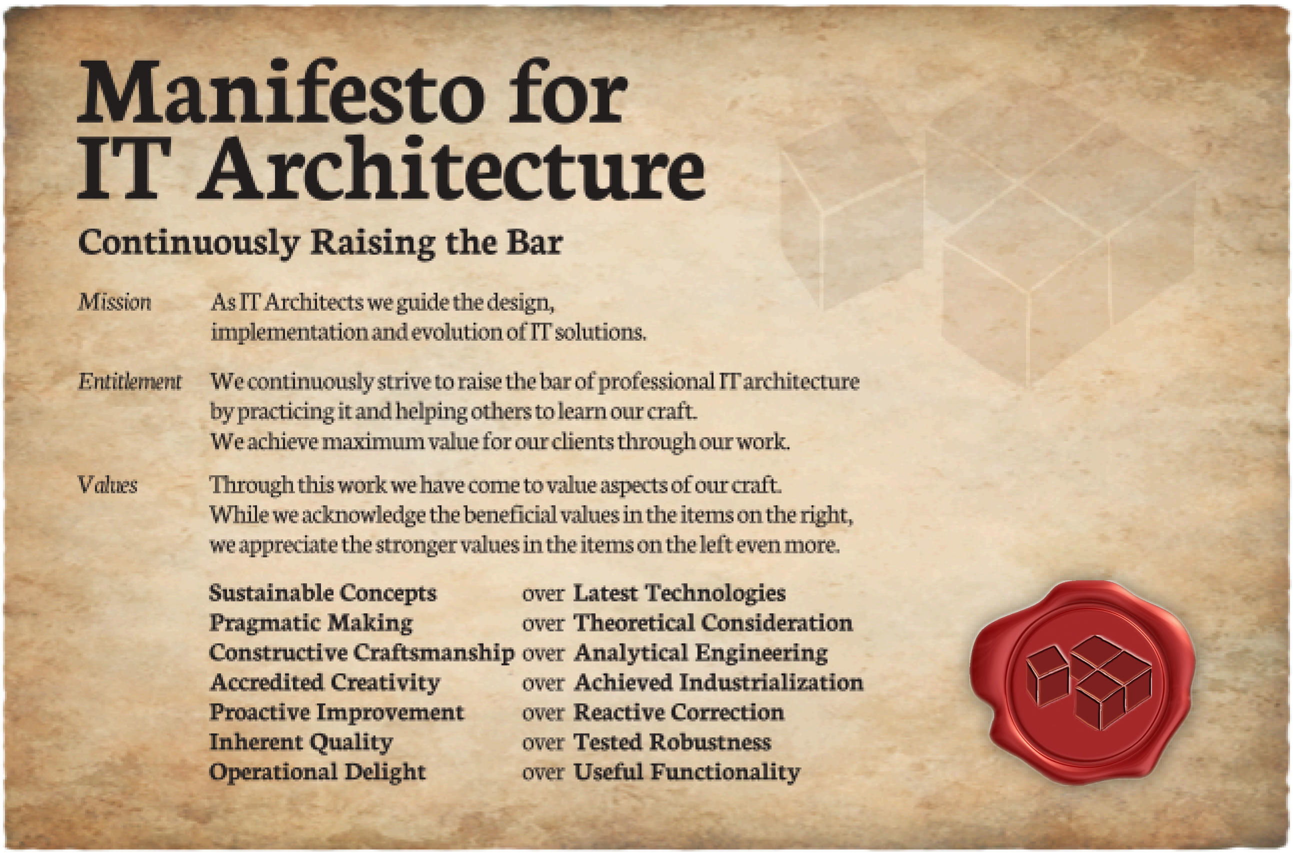


(Software) **Architecture** gilt als die “Königsdisziplin” im **Software Engineering**, da sie das zentrale, allgemeine und konzeptuelle Bindeglied zwischen den vielen, potentiellen, spezifischen, realisierten **Fachlichkeiten** und den vielen, potentiellen, spezifischen, realisierenden **Technologien** ist. Die architekturelle Konstruktion einer Anwendung findet im logischen Schritt “Design” innerhalb des BizDevOps-Workflows im Software Engineering statt.

## Fragen

- ? Wieso gilt **Architektur** als die “Königsdisziplin” im Software Engineering?

# Architecture Manifesto



Manifesto for
IT Architecture

Continuously Raising the Bar

*Mission* As IT Architects we guide the design,
implementation and evolution of IT solutions.

*Entitlement* We continuously strive to raise the bar of professional IT architecture
by practicing it and helping others to learn our craft.
We achieve maximum value for our clients through our work.

*Values* Through this work we have come to value aspects of our craft.
While we acknowledge the beneficial values in the items on the right,
we appreciate the stronger values in the items on the left even more.

**Sustainable Concepts** over **Latest Technologies**
**Pragmatic Making** over **Theoretical Consideration**
**Constructive Craftsmanship** over **Analytical Engineering**
**Accredited Creativity** over **Achieved Industrialization**
**Proactive Improvement** over **Reactive Correction**
**Inherent Quality** over **Tested Robustness**
**Operational Delight** over **Useful Functionality**

Das **Manifesto for IT Architecture** ist eine Grundsatzerklärung für IT-Architektur. Sie besagt zuallererst, daß kontinuierlich die Meßlatte zu erhöhen ist (“Continuously Raising the Bar”), da nach gerade mal 50 Jahren Software Engineering und Software Architecture wir zwar bereits etliche Best Practices kennen, aber die Disziplin sich sicherlich noch sehr lange weiterentwickeln muss.

Der Auftrag (Mission) an IT-Architekten ist, die Konstruktion, die Implementierung und die Evolution von IT-Lösungen zu leiten. Der Anspruch (Entitlement) ist es, dabei die Messlatte kontinuierlich zu erhöhen und anderen zu helfen, das “Handwerk” zu erlernen. Selbstredend ist die Tatsache, daß durch die Arbeit von Architekten maximaler Mehrwert für die Kunden erzielt wird.

Die Basis-Werte (Values), welche in diesem Handwerk eine zentrale Rolle spielen und sehr geschätzt werden, sind: **Latest Technologies**, **Theoretical Consideration**, **Analytical Engineering**, **Achieved Industrialization**, **Reactive Correction**, **Tested Robustness** und **Useful Functionality**.

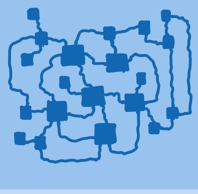
Darüber hinaus gibt es aber zusätzliche Werte, welche ebenfalls eine zentrale Rolle spielen und noch sehr viel mehr geschätzt werden: **Sustainable Concepts** (der Inhalt von **Architecture Fundamentals**!), **Pragmatic Making**, **Constructive Craftsmanship**, **Accredited Creativity**, **Proactive Improvement**, **Inherent Quality** und **Operational Delight**.

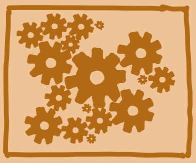
## Fragen

- ? (keine)

# Complex vs. Complicated

| | **complex** | **complicated** |
|---|---|---|
| FOCUS | refers to the **extrinsic** and **higher**- or **macro**-level difficulty of a system, | refers to the **intrinsic** and **lower**- or **micro**-level difficulty of a system, |
| RATIONALE | because the system involves many different and **connected** parts | because the system involves many different and **difficult** aspects |
| CHALLENGE | which take time to **comprehend** and **master** in **total**, | which take time to **understand** and **learn** in **detail**, |
| INSIGHT | and which nevertheless are **easy** to explain. | and which usually are **hard** to explain. |





| | complex | complicated |
|---|---|---|
| NOTICE | **Simple** (non-complicated) systems can be **complex**. | **Clear** (non-complex) systems can be **complicated**. |
| RECOGNIZE | **Architecture** primarily has to master the **complex** aspects of a system. | **Development** primarily has to master the **complicated** aspects of a system. |

“Komplex” bezieht sich auf die extrinsische Schwierigkeit eines Systems auf der höheren Ebene bzw. der Makro-Ebene, da das System aus vielen verschiedenen, miteinander verbundenen Teilen besteht. Es braucht deshalb Zeit, um dieses zu begreifen und als Ganzes zu meistern, obwohl jedes Teil des Systems meist leicht zu erklären ist.

“Kompliziert” bezieht sich auf die intrinsische Schwierigkeit eines Systems auf der unteren Ebene bzw. der Mikro-Ebene, da das System viele verschiedene, schwierige Aspekte umfasst. Es braucht deshalb Zeit, um die Teile im Detail zu verstehen und zu lernen, und jedes Teil des Systems ist meist auch schwer zu erklären.

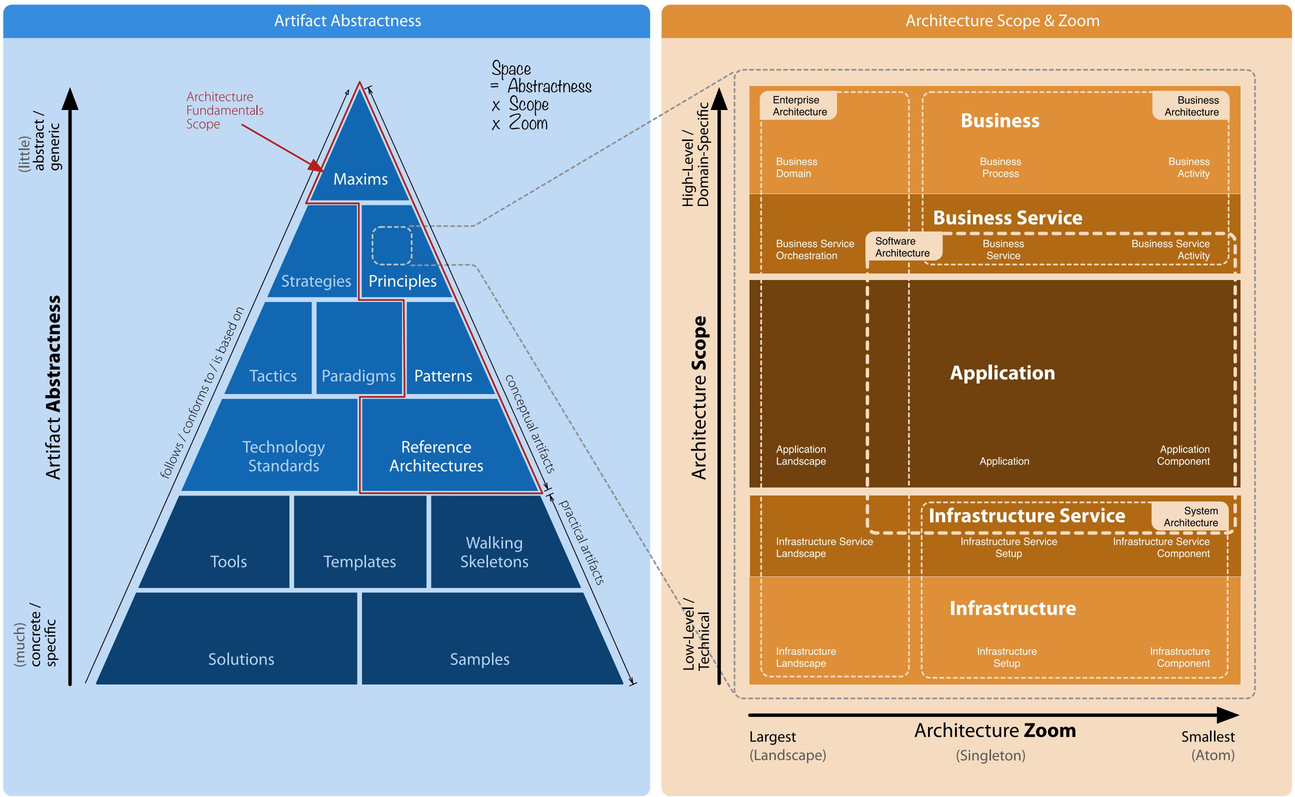
Beachte: Einfache (nicht-komplizierte) Systeme können komplex sein – offensichtliche (nicht-komplexe) Systeme können kompliziert sein.

Der entscheidende Unterschied ist: Die Architektur bzw. die Konstruktion muss vor allem die komplexen Aspekte eines Systems meistern. Die Entwicklung bzw. die Realisierung muss vor allem die komplizierten Aspekte eines Systems meistern.

## Fragen

- ? Muss sich **Architektur** primär um **komplexe** oder **komplizierte** Aspekte eines Systems kümmern?

# Architecture Space



Der **IT Architecture Space** besteht aus drei Dimensionen: der **Artifact Abstractness** (einem Abstraktheitsgrad aller Artefakte, die der Architekt kennt), dem **Architecture Scope** (dem Bereich von Architektur, in dem man agiert) und dem **Architecture Zoom** (der Detailstufe von Architektur, in der man agiert).

Man unterscheidet primär drei Architecture Scopes: (high-level/domain-specific) **Business**, **Application** und (low-level/technical) **Infrastructure**. Zusätzlich werden die zwei sekundären Architecture Scopes **Business Service** und **Infrastructure Service** genutzt, um den "gedanklichen Sprung" von Business zu Application und von Application zu Infrastructure in der Praxis deutlich kleiner und sinnvoller ausfallen zu lassen.

Auf jedem Architecture Scope kann man mindestens drei verschiedene **Architecture Zooms** unterscheiden: von **Landscape** (largest), über **Singleton** bis zu **Atom** (smallest).

Im Raum aus Architecture Scope und Zoom unterscheidet man vier Arten von **IT Architecture**: **Business Architecture** (Facharchitektur), **Software Architecture** (Software-Architektur), **System Architecture** (System-Architektur) und **Enterprise Architecture** (Unternehmens-Architektur).

Je höher/niedriger der Abstraktheitsgrad der Artefakte, umso größer/kleiner ist üblicherweise die Abdeckung des Raums aus Architecture Scope und Architecture Zoom.

## Fragen

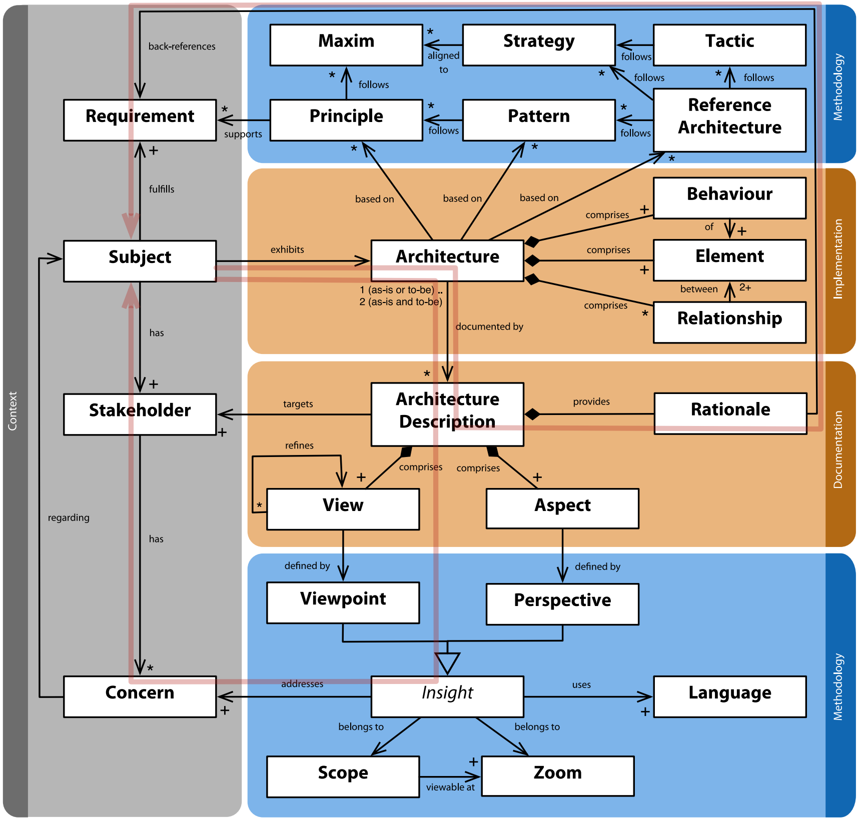
- ? Aus welchen drei Dimensionen besteht der IT Architecture Space?
- ? Welche vier Arten von Architektur kennt man in der IT?

Intellectual Content: Version 1.0.9 (2020-06-23), Authored 2011-2020 by Dr. Ralf S. Engelschall and Marc Mährländer, based on pre-work of msg Applied Technology Research
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

ARCHITECTURE
FUNDAMENTALS

# Architecture Ontology

TECHNISCHE
UNIVERSITÄT
MÜNCHEN



**Requirement:** A functional or non-functional demand or imposed obligation on the **Subject**.

**Subject:** Any type of business process, business service, software application, infrastructure service or infrastructure setup.

**Stakeholder:** Person, group or entity with an interest in or **Concerns** about the **Architecture**.

**Concern:** Requirement, objective, intention, or aspiration a **Stakeholder** has on an **Architecture**.

**Architecture Description:** Set of artifacts that document an **Architecture** in a way **Stakeholders** can understand and ensures their **Concerns** are met.

**Viewpoint:** Collection of templates and conventions for constructing one type of **View**. It addresses **Concerns** and contains guidelines for constructing a View.

**Perspective:** Collection of decisions, guidelines and rules that ensure a **Subject** exhibits a set of non-functional **Requirements**, considered across a number of **Views**.

**Insight:** Superordinate abstract concept to address particular **Concerns** through **Viewpoints** and **Perspectives**.

**Architecture:** Inherent static and dynamic structure of a **Subject** which comprise **Elements**, the visible **Behaviour** of **Elements** and **Relationships** between **Elements**.

**Element:** Fundamental part from which a **Subject** can be considered to be constructed.

**Behaviour:** Run-time characteristic of **Architecture Elements** of the **Subject**.

**Relationship:** Static or dynamic relationship between **Elements** of an **Architecture** of the **Subject**.

**Rationale:** Fundamental reasons for a particular chosen **Architecture**, usually strongly based on non-functional **Requirements**.

**View:** Representation of one or more structural aspects of an **Architecture** that illustrates how the **Architecture** addresses one or more **Concerns**.

**Aspect:** Representation of one or more non-functional aspects of an **Architecture** that illustrate how the **Architecture** addresses one or more **Concerns**.

**Language:** Formal, semi-formal or even prose language to describe an Insight, so it optimally addresses the **Concerns** of **Stakeholders**.

**Maxim:** Fundamental, generally valid set of values and rules to guide the architecture discipline (think: law).

**Strategy:** Long-term situation-independent plan of approaches to achieve a particular goal (think: war). Aligned and not in conflict with any **Maxims**.

**Tactic:** Short-term situation-dependent plan of actions to achieve a particular goal (think: battle). Following and supporting a **Strategy**.

**Principle:** Fundamental truth, rule, tenet or policy an **Architecture** follows.

**Pattern:** Proven recurring theme, structure, approach or behavior an **Architecture** and its **Elements** can follow.

**Reference Architecture:** Reusable proven Architecture template, **based** on a set of Patterns and following one or more **Tactics** and **Strategies**.

**Scope:** Primary area of the **Architecture** space an **Insight** addresses: either Business, Business Service, Application, Infrastructure Service or Infrastructure.

**Zoom:** Level of view distance to the **Architecture** an **Insight** has. Also known as the **Insight** detail level, ranging from smallest atoms, over singletons to the largest landscape.

Intellectual Content: Version 1.1.8 (2020-08-13), Authored 2010-2020 by Dr. Ralf S. Engelschall, inspired by ISO/IEC/IEEE 42010:2011, IEEE 1471:2000 and The Open Group TOGAF 9
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Damit Architekten sinnvoll in der Praxis kommunizieren können, muss man sich auf ein paar wenige Grundbegriffe und deren Bedeutung einigen. Die Begriffe werden in einer Taxonomie definiert und in der **Architecture Ontology** in Beziehung zueinander gesetzt.

In der Architecture Ontology gibt es vor allem zwei wichtige “Schleifen”. Beide starten beim **Subject**, welches eine **Architecture** hat, welche über die **Architecture Description** (im Deutschen “IT-Konzept”) beschrieben wird.

Schleife 1: Die Architecture Description liefert **Rationales** (Begründungen) für Entscheidungen, welche im Idealfall auf **Requirements** zurückreferenzieren. Denn eine Architecture Description soll nicht das **WAS**, sondern vor allem das **WARUM** dokumentieren. Denn das WAS sieht man auch im Code, aber das WARUM nicht!

Schleife 2: Die Architecture Description besteht aus **Views** (Sichten) und **Aspects** (Aspekten), welche methodisch **Viewpoints** und **Perspectives** genannt werden. Beide zusammen liefern **Insights** (Einblicke) auf **Scope** und **Zoom Level** (siehe Architecture Space!) und werden über eine bestimmte (graphische oder textuelle) **Language** dokumentiert. In jedem Fall werden nur solche Insights gegeben, welche einen **Concern** (Anliegen) eines **Stakeholders** (Interessenvertreter) adressieren. Denn man programmiert auch nichts, was nicht gebraucht wird!

## Fragen

- ? Was sollte eine **Architecture Description** (IT-Konzept) neben dem **WAS** vor allem dokumentieren?
- ? Was sollte eine **Architecture Description** (IT-Konzept) über **Insights** (Einblicke) vor allem adressieren?

ARCHITECTURE
FUNDAMENTALS

# Requirements Basics

TECHNISCHE
UNIVERSITÄT
MÜNCHEN

Intellectual Content: Version 1.0.2 (2022-10-28), Authored 2018-2022 by Dr. Ralf S. Engelschall, based on ideas from BABOK 2.0:2009, IEEE Std 610:1990 and IEEE Std 830–1998
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

## Requirements Specification

A binding document that specifies the requirements for a solution, by focusing on the WHAT and WHY of the solution — and *not* giving instructions for the HOW.

The documented set of requirements has to be: correct, unambiguous, complete, consistent, ranked, verifiable, modifiable, and traceable.

## Requirement Classes

**FR** **Functional** (Shall Do)

A condition or capability that a solution must have to provide its service in terms of its behaviour and information. Think: Functionality.

**NFR** **Non-Functional** (Shall Be)

A condition, property or quality that a solution must have to satisfy a contract, standard, or other formally imposed obligation. Think: Constraints and "*-ilities".

## Requirement Interdependencies

**POS** **Positive** (Backing)

One requirement supports the other (e.g. for NFRs: Maintainability and Comprehensibility usually support Adaptability, Portability, Modifiability, etc., and Scalability usually supports Availability, etc.)

**NEG** **Negative** (Trade-Off)

One requirement interferes with the other (e.g. for NFRs: Security usually interferes with Efficiency, Usability, Performance, etc., and Orthogonality can interfere with Usability)

## Requirement Characteristics

**S** **Specific**

The requirement is precise, unambiguous, and clear on what should be done.

**M** **Measurable**

The requirement can be verified when it has been achieved by use of a particular test.

**A** **Achievable**

The requirement is achievable given existing circumstances and feasible and viable solutions.

**R** **Relevant**

The requirement is relevant to the goals of the context.

**T** **Time-Bound**

The requirement can be achieved within a reasonable time frame.

## Requirement Life-Time

**E** **Enduring**

The requirement lasts forever, as it is derived from core activities and organisational structures.

**V** **Volatile**

The requirement can be temporary, as it might change over time.

## Requirement Expression

[<req-id>] <req-name>**:**
<subject/actor>
**SHALL**
<result/action/condition>
**BECAUSE**
<rationale>

Die **Requirements Specification** (auch Anforderungsspezifikation, Fachkonzept oder Pflichtenheft) ist ein bindendes Dokument, in dem primär das WAS und das WARUM der Lösung spezifiziert wird, allerdings nicht das konkrete technische WIE. Die Menge an Anforderungen muss korrekt, eindeutig, vollständig, konsistent, priorisiert, nachprüfbar, änderbar und nachvollziehbar sein.

Es gibt zwei Arten von Requirements: **Functional Requirement** ("Shall Do", Funktionalität) und **Non-Functional Requirements** ("Shall Be", Bedingungen, im Englischen oft über die Wörter mit dem Ende "-ility" ausgedrückt). Der Architekt kümmert sich primär um letztere.

Requirements können außerdem sich gegenseitig positiv (Backing) oder negativ (Trade-Off) beeinflussen. Der Architekt kümmert sich auch hier primär um letztere.

Requirements sollten "SMART" sein: **Specific**, **Measurable**, **Achievable**, **Relevant** und **Time-Bound**.

Außerdem sind Requirements entweder **Enduring** (feststehend) oder **Volatile** (unbeständig). Der Architekt sollte auf letztere achten.

## Fragen

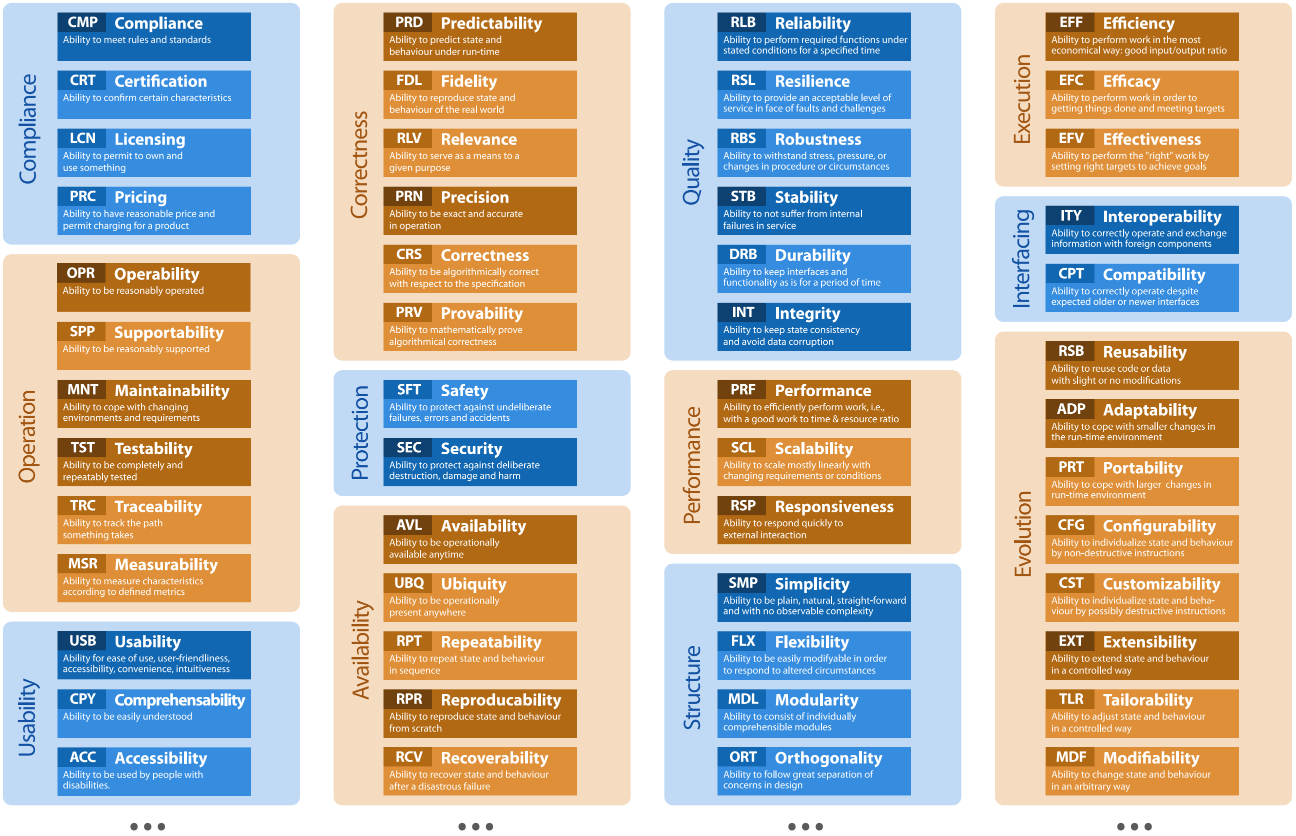
? Welche Art von **Requirements** sollte der Architekt primär im Auge haben und sollten von ihm explizit in der Lösungsfindung adressiert werden?

ARCHITECTURE
FUNDAMENTALS

# Non-Functional Requirements

TECHNISCHE
UNIVERSITÄT
MÜNCHEN

AF 03.2



### Compliance

**CMP Compliance**
Ability to meet rules and standards

**CRT Certification**
Ability to confirm certain characteristics

**LCN Licensing**
Ability to permit to own and use something

**PRC Pricing**
Ability to have reasonable price and permit charging for a product

### Operation

**OPR Operability**
Ability to be reasonably operated

**SPP Supportability**
Ability to be reasonably supported

**MNT Maintainability**
Ability to cope with changing environments and requirements

**TST Testability**
Ability to be completely and repeatably tested

**TRC Traceability**
Ability to track the path something takes

**MSR Measurability**
Ability to measure characteristics according to defined metrics

### Usability

**USB Usability**
Ability for ease of use, user-friendliness, accessibility, convenience, intuitiveness

**CPY Comprehensability**
Ability to be easily understood

**ACC Accessibility**
Ability to be used by people with disabilities.

•••

### Correctness

**PRD Predictability**
Ability to predict state and behaviour under run-time

**FDL Fidelity**
Ability to reproduce state and behaviour of the real world

**RLV Relevance**
Ability to serve as a means to a given purpose

**PRN Precision**
Ability to be exact and accurate in operation

**CRS Correctness**
Ability to be algorithmically correct with respect to the specification

**PRV Provability**
Ability to mathematically prove algorithmical correctness

### Protection

**SFT Safety**
Ability to protect against undeliberate failures, errors and accidents

**SEC Security**
Ability to protect against deliberate destruction, damage and harm

### Availability

**AVL Availability**
Ability to be operationally available anytime

**UBQ Ubiquity**
Ability to be operationally present anywhere

**RPT Repeatability**
Ability to repeat state and behaviour in sequence

**RPR Reproducability**
Ability to reproduce state and behaviour from scratch

**RCV Recoverability**
Ability to recover state and behaviour after a disastrous failure

•••

### Quality

**RLB Reliability**
Ability to perform required functions under stated conditions for a specified time

**RSL Resilience**
Ability to provide an acceptable level of service in face of faults and challenges

**RBS Robustness**
Ability to withstand stress, pressure, or changes in procedure or circumstances

**STB Stability**
Ability to not suffer from internal failures in service

**DRB Durability**
Ability to keep interfaces and functionality as is for a period of time

**INT Integrity**
Ability to keep state consistency and avoid data corruption

### Performance

**PRF Performance**
Ability to efficiently perform work, i.e., with a good work to time & resource ratio

**SCL Scalability**
Ability to scale mostly linearly with changing requirements or conditions

**RSP Responsiveness**
Ability to respond quickly to external interaction

### Structure

**SMP Simplicity**
Ability to be plain, natural, straight-forward and with no observable complexity

**FLX Flexibility**
Ability to be easily modifyable in order to respond to altered circumstances

**MDL Modularity**
Ability to consist of individually comprehensible modules

**ORT Orthogonality**
Ability to follow great separation of concerns in design

•••

### Execution

**EFF Efficiency**
Ability to perform work in the most economical way: good input/output ratio

**EFC Efficacy**
Ability to perform work in order to getting things done and meeting targets

**EFV Effectiveness**
Ability to perform the "right" work by setting right targets to achieve goals

### Interfacing

**ITY Interoperability**
Ability to correctly operate and exchange information with foreign components

**CPT Compatibility**
Ability to correctly operate despite expected older or newer interfaces

### Evolution

**RSB Reusability**
Ability to reuse code or data with slight or no modifications

**ADP Adaptability**
Ability to cope with smaller changes in the run-time environment

**PRT Portability**
Ability to cope with larger changes in run-time environment

**CFG Configurability**
Ability to individualize state and behaviour by non-destructive instructions

**CST Customizability**
Ability to individualize state and behaviour by possibly destructive instructions

**EXT Extensibility**
Ability to extend state and behaviour in a controlled way

**TLR Tailorability**
Ability to adjust state and behaviour in a controlled way

**MDF Modifiability**
Ability to change state and behaviour in an arbitrary way

•••

Intellectual Content: Version 1.0.9 (2020-08-13), Authored 2009-2020 by Dr. Ralf S. Engelschall, based on ISO/IEC FDIS 9126-1:2000
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

**Non-Functional Requirements** gibt es potenziell sehr viele. Für jede Lösung muss deshalb erst die tatsächliche Menge an solchen Requirements festgestellt werden. Diese Menge ist unbedingt vom Architekten zu minimieren!

Bei jedem vertraglich festgeschriebenen Requirement sollte darauf geachtet werden, daß es klar definiert ist, da es große Ähnlichkeiten zwischen Requirements gibt und die Unterschiede teilweise sehr subtil sind.

Ein paar der in der Praxis fast immer zu berücksichtigenden Non-Functional Requirements sind **Maintainability**, **Usability**, **Security**, **Availability**, **Reliability**, **Performance**, **Responsiveness** und **Adaptability**.

## Fragen

? Nennen Sie 3 in der Praxis häufig zu berücksichtigende **Non-Functional Requirements**!